

MosicaFE

统一调用 FEM、VEM 与 RT0 求解偏微分方程的 Python 库
使用手册与开发指南

MosicaFE contributors

2026 年 9 月 20 日

摘要. 本手册介绍 MosicaFE——一个把有限元 (FEM)、虚拟元 (VEM) 与最低阶 Raviart-Thomas 混合元 ($RT_0 \times P_0$) 统一到同一套接口下的 Python 库。用户只需在主文件里写几行代码: 选择网格、选择“元”、选择线性求解器、选择可视化工具, 即可离散并求解 Poisson 型偏微分方程。库的分层设计 (`MosicaFE.core`、`MosicaFE.spaces`、`MosicaFE.physics`、`MosicaFE.solvers`、`MosicaFE.postprocess`) 使得新增一种单元或 新增一门方程都是局部改动。手册给出完整的调用示例、数学背景、精度与收敛性验证, 以及扩展框架的模板。

目录

第一章 引言	1
1.1 概述	1
1.2 设计哲学	1
1.3 快速开始	2
1.4 包结构	3
1.5 本手册的组织	3
第二章 安装与环境	5
2.1 系统要求	5
2.1.1 Python 版本	5
2.1.2 依赖	5
2.2 安装	5
2.2.1 方式一：以包的形式安装（推荐）	5
2.2.2 方式二：直接使用源码	5
2.3 验证安装	6
2.4 编译本手册	6
第三章 数学背景	7
3.1 模型问题	7
3.2 弱形式	7
3.3 有限元离散（FEM）	7
3.4 虚拟元离散（VEM）	8
3.4.1 投影算子	8
3.4.2 稳定化项	9
3.4.3 载荷向量与质量矩阵	9
3.5 $RT_0 \times P_0$ 混合离散	10
3.6 误差估计	10
第四章 核心模块：网格与求积	11
4.1 网格数据结构	11
4.1.1 网格的组成部分	11

4.1.2	facet 拓扑	11
4.1.3	面拓扑与定向: <code>core/facets.py</code>	12
4.2	网格生成	13
4.3	网格属性与几何量	13
4.4	网格加密	14
4.5	参考单元	14
4.6	求积规则	15
第五章	离散空间	17
5.1	空间抽象	17
5.2	Lagrange 元 (经典有限元)	17
5.2.1	P1 三角形	17
5.2.2	P2 三角形	18
5.2.3	张量积元	18
5.2.4	自由度的全局编号	18
5.3	虚拟元空间	19
5.4	RT0×P0 混合空间	19
5.4.1	空间层给出的两个块	20
5.5	空间工厂 <code>FESpace</code>	21
第六章	物理问题与组装	22
6.1	问题接口	22
6.2	Poisson 问题	22
6.3	边界条件	23
6.3.1	强加 Dirichlet 条件的正确做法	23
6.3.2	其它边界条件	23
6.4	组装算法	24
6.5	使用 Poisson 问题	24
6.6	PDE 注册表	25
第七章	线性求解器	26
7.1	求解器接口	26
7.2	直接法	26
7.3	迭代法	27
7.4	求解器选择指南	27
第八章	误差分析与收敛性	28
8.1	误差范数	28
8.2	收敛性研究框架	28
8.3	收敛阶估计	29

8.4 实测验证结果	29
第九章 可视化	31
9.1 可视化工具一览	31
9.2 画解	31
9.3 画收敛曲线	32
第十章 完整示例	33
10.1 示例一：两行求解 Poisson 方程	33
10.2 示例二：换一种元（同一段代码，三种方法）	33
10.3 示例三：多边形网格上的虚拟元	34
10.4 示例四：自定义新 PDE (Helmholtz)	35
10.5 示例五：反应扩散方程	36
第十一章 进阶话题	37
11.1 精度陷阱与经验教训	37
11.1.1 陷阱一：Dirichlet 消去漏掉右端项修正	37
11.1.2 陷阱二：张量积单元的求积阶不足	37
11.1.3 陷阱三：虚拟元投影的常数模式规范化	37
11.1.4 陷阱四：载荷 / 质量矩阵的几何权重	37
11.1.5 陷阱五：numpy 的重复下标赋值	38
11.2 网格细化策略	38
11.3 性能与向量化	38
11.4 与其它库的关系	38
第十二章 扩展框架	40
12.1 新增一种单元	40
12.2 新增一门方程	41
12.3 新增一种混合格式（以 $RT_0 \times P_0$ 为模板）	42
12.4 新增一种网格	42
12.5 开发流程与测试要求	43
第十三章 常见问题与排查	44
13.1 常见问题	44
13.1.1 导入错误	44
13.1.2 收敛阶不对	44
13.1.3 迭代法不收敛	44
13.1.4 内存不足	45
13.2 常见问答	45

附录 A API 速查	46
A.1 顶层模块 MosicaFE.	46
A.2 Mesh	47
A.3 FESpace 与空间方法	47
A.4 物理问题	48
A.5 求解器与后处理	49
附录 B 数学符号	50
B.1 一般记号	50
B.2 函数空间	50
B.3 算子与范数	50
附录 C 参考文献	52

第一章 引言

1.1 概述

MosicaFE 是面向教学与科研的偏微分方程求解库。它把原本互相独立的三套程序（经典有限元、虚拟元、RT0 混合有限元）整合成一个包、一套接口，并使用同一套网格、求解器与后处理工具。

库的核心设计目标是：

- (1) 调用极简：主文件通常不超过 10 行；
- (2) 模块解耦：网格、单元、方程、线性代数、后处理各自独立；
- (3) 易于扩展：新增单元或新增 PDE 都只需实现少量接口；
- (4) 可验证：内置收敛性研究与误差范数，收敛阶可一键复现；
- (5) 自包含：只依赖 numpy (scipy 与 matplotlib 可选)。

1.2 设计哲学

MosicaFE 的设计对应有有限元方法的数学结构：

$$\underbrace{\text{PDE}}_{\text{physics}} \xrightarrow{\text{弱形式}} \underbrace{\text{变分问题}}_{\text{spaces}} \xrightarrow{\text{离散化}} \underbrace{Au = b}_{\text{assembly}} \xrightarrow{\text{求解}} \underbrace{u_h}_{\text{postprocess}} \quad (1.1)$$

每一步由独立的模块负责，模块之间只通过明确的接口通信：

- `MosicaFE.core` 提供网格与求积规则；
- `MosicaFE.spaces` 把”选哪种单元”翻译成矩阵级接口：刚度矩阵、质量矩阵、载荷向量（详见第五章）；
- `MosicaFE.physics` 只做这些矩阵的线性组合，因此描述一门新 PDE 通常只要十几行；
- `MosicaFE.solvers` 负责线性代数；
- `MosicaFE.postprocess` 负责误差、收敛性与可视化。

要点

”元”与”方程”的正交分解是 MosicaFE 与三套原始程序最大的区别。原程序把”三角形网格 + 线性元 + Poisson + `spsolve` + 画图”写死在同一个文件里；现在这几件事互不相干：换一种元不必改方程，换一门方程不必改单元。

1.3 快速开始

下面是一个完整的 2D Poisson 求解程序：

```
1 import numpy as np
2 from MosicaFE import Mesh, solve_poisson
3
4 source = lambda x: 2*np.pi**2 * np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])
5
6 u = solve_poisson(Mesh.rectangle(nx=32, ny=32), element="P2", f=source)
7 print("deg of freedom:", u.size)
```

Listing 1.1: 示例 1.1: 两行求解 Poisson 方程

等价的”分步”写法（也是本手册后续使用的标准形式）：

```
1 import numpy as np
2 from MosicaFE import Mesh, FESpace, PoissonProblem
3 from MosicaFE.postprocess import plot_solution
4
5 def exact(x):
6     return np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])
7
8 def source(x):
9     return 2*np.pi**2 * exact(x)
10
11 mesh = Mesh.rectangle(nx=32, ny=32)           # (1) 选网格
12 V = FESpace.lagrange(mesh, degree=2)         # (2) 选元
13 problem = PoissonProblem(V, f=source, g=lambda x: 0.0)
14 u = problem.solve(method="direct")            # (3) 选求解器
15 plot_solution(u, mesh, V, filename="p2.png")  # (4) 选可视化
16
17 print(problem.compute_errors(u, exact))
```

Listing 1.2: 示例 1.2: 选择网格 / 元 / 求解器 / 可视化

把第 1.2 行中的 `FESpace.lagrange(mesh, degree=2)` 换成 `FESpace.vem(mesh)`，得到的就不再是有限元，而是同一个方程在同一种网格上的虚拟元解；程序其它部分一字不改。

要点

这就是本库存在的意义：让”换方法”的代价降到一行代码。

1.4 包结构

表 1.1: MosicaFE 的模块划分

模块	职责
MosicaFE.core	网格数据结构与生成（区间 / 矩形 / 长方体 / 多边形 / Voronoi）、面拓扑与定向（ <code>facets.py</code> , $H(\text{div})$ 协调空间的基础）、求积规则、几何工具（面积、体积、雅可比、梯度变换）
MosicaFE.spaces	离散空间： <code>LagrangeSpace</code> （P1/P2/Q1/Q2）、 <code>VEMSpace</code> （ $k = 1$ 虚拟元）、 <code>RT0Space</code> （ $\text{RT0} \times \text{P0}$ ），以及工厂 <code>FESpace</code>
MosicaFE.physics	PDE 弱形式与组装： <code>PoissonProblem</code> 、 <code>MixedPoissonProblem</code> 、 <code>ReactionDiffusionProblem</code> ，含 PDE 注册表
MosicaFE.solvers	线性求解器：直接法、CG、GMRES、BiCGSTAB、MINRES 与自动选择
MosicaFE.postprocess	误差范数（ L^2 / H^1 / L^∞ ）、收敛性研究、可视化、混合格式（压力 + 通量）的误差范数

要点

FEM、VEM、RT0 三类元全部是库自身的实现：RT0×P0 的显式基函数、精确质量矩阵、面拓扑与鞍点组装分别位于 `spaces/rt0.py`、`core/facets.py` 与 `physics/mixed_poisson.py`，MosicaFE 不依赖、也不需要接入任何外部有限元后端。

1.5 本手册的组织

- 第二章：安装与环境；
- 第三章：三种方法的数学背景；
- 第四–五章：网格、求积与离散空间；
- 第六–七章：方程组装与线性求解；
- 第八–九章：误差分析与可视化；
- 第十章：完整示例；

- 第十一章-十二章：进阶话题与框架扩展；
- 第十三章：常见问题；
- 附录 A-C：API 速查、数学符号、参考文献。

第二章 安装与环境

2.1 系统要求

2.1.1 Python 版本

要求 Python ≥ 3.9 。开发与验证使用的环境为 Python 3.12.10 (conda 环境 `fealpy`), `numpy` 2.2.4、`scipy` 1.15.2、`matplotlib` 3.10.1。

2.1.2 依赖

表 2.1: 依赖关系

包	是否必需	用途
<code>numpy</code>	必需	数组、稠密线性代数
<code>scipy</code>	可选	稀疏矩阵、 <code>spsolve</code> 、Krylov 求解器、Voronoi 网格
<code>matplotlib</code>	可选	可视化

缺少 `scipy` 时, MosicaFE 自动退化为纯 `numpy` 的稠密实现 (`MosicaFE._compat` 统一处理), 小规模问题照常可解; 缺少 `matplotlib` 时, 所有绘图函数打印一条提示并返回 `None`, 不会报错。

2.2 安装

2.2.1 方式一: 以包的形式安装 (推荐)

```
cd D:\mosica\MosicaFE
python -m pip install -e .
```

2.2.2 方式二: 直接使用源码

把 MosicaFE 的父目录加入模块搜索路径即可, 无需安装:

```
# PowerShell
cd D:\mosica
```

```
$env:PYTHONPATH = "D:\mosica"
& "D:\anaconda\envs\fealpy\python.exe" MosicaFE\examples\poisson_demo.py
```

2.3 验证安装

MosicaFE 自带两个不依赖第三方测试框架的脚本：

```
& "D:\anaconda\envs\fealpy\python.exe" MosicaFE\tests\run_tests.py
& "D:\anaconda\envs\fealpy\python.exe" MosicaFE\tests\run_checks.py
```

前者跑 54 个单元/集成测试，后者打印各方法的实测收敛阶。预期输出片段：

```
=== P2 ===
  level      h          L2    rate      H1    rate      Linf    rate
    0 0.35355  3.4643e-02    - 4.4391e-01    - 1.4645e-01    -
    1 0.17678  4.4076e-03  2.97 1.1632e-01  1.93 3.8060e-02  1.94
    2 0.08839  5.5040e-04  3.00 2.9416e-02  1.98 9.6074e-03  1.99
    3 0.04419  6.8699e-05  3.00 7.3752e-03  2.00 2.4076e-03  2.00
rate[L2] (last 3) = 3.002
rate[H1] (last 3) = 1.990
```

如果安装了 `pytest`，也可以直接：

```
python -m pytest MosicaFE/tests -q
```

2.4 编译本手册

本文档使用 `ctexbook` 文档类，正文含中文，需要用 `XeLaTeX` 编译两遍：

```
xelatex MosicaFE_Guidebook.tex
xelatex MosicaFE_Guidebook.tex
```

在 `TeXShop` 中把引擎选为 `XeLaTeX` 并点击 `Typeset` 即可。

第三章 数学背景

3.1 模型问题

本库（以及原来的三套程序）求解的核心模型问题是 Poisson 方程

$$\begin{cases} -\nabla \cdot (\kappa \nabla u) = f, & \text{in } \Omega, \\ u = g, & \text{on } \partial\Omega, \end{cases} \quad (3.1)$$

其中 $\Omega \subset \mathbb{R}^d$ ($d = 1, 2, 3$) 是有界区域, $\kappa > 0$ 是扩散系数。当 $g = 0$ 且 $\kappa = 1$ 时即经典的 $-\Delta u = f$ 。

3.2 弱形式

定义 3.1 (弱形式). 记 $V = \{v \in H^1(\Omega) : v|_{\partial\Omega} = 0\}$, $V_g = \{v \in H^1(\Omega) : v|_{\partial\Omega} = g\}$ 。求 $u \in V_g$ 使得

$$a(u, v) = \ell(v), \quad \forall v \in V, \quad (3.2)$$

其中

$$a(u, v) = \int_{\Omega} \kappa \nabla u \cdot \nabla v \, dx, \quad \ell(v) = \int_{\Omega} f v \, dx. \quad (3.3)$$

ℓ 也可以带 Neumann 数据项 $\int_{\Gamma_N} h v \, ds$, MosicaFE 的物理层接口为此预留了扩展点。

3.3 有限元离散 (FEM)

设 $V_h \subset V$ 是有限维子空间, 基函数为 $\{\varphi_i\}_{i=1}^n$ 。离散问题为: 求 $u_h = \sum_j u_j \varphi_j$ 使得

$$\sum_{j=1}^n u_j a(\varphi_j, \varphi_i) = \ell(\varphi_i), \quad i = 1, \dots, n, \quad (3.4)$$

即线性方程组

$$Au = b, \quad A_{ij} = \int_{\Omega} \kappa \nabla \varphi_j \cdot \nabla \varphi_i \, dx, \quad b_i = \int_{\Omega} f \varphi_i \, dx. \quad (3.5)$$

在参考单元 $\hat{K}$ 上计算时用到梯度变换

$$\nabla_x \varphi_i = J_K^{-\top} \nabla_{\hat{x}} \hat{\varphi}_i, \quad \int_K g \, dx = \int_{\hat{K}} g \circ F_K |\det J_K| \, d\hat{x}. \quad (3.6)$$

表 3.1: MosicaFE 中 Lagrange 元的信息

单元	名字	每个单元的局部自由度	自由度类型
三角形	P1	3	顶点
三角形	P2	6	顶点 + 棱中点
四边形	Q1	4	顶点
四边形	Q2	9	顶点 + 棱中点 + 单元中心
四面体	P1	4	顶点
四面体	P2	10	顶点 + 棱中点
六面体	Q1	8	顶点
六面体	Q2	27	顶点 + 棱 + 面 + 体

3.4 虚拟元离散 (VEM)

虚拟元方法的关键思想是：不需要写出基函数的显式表达式。一次 ($k = 1$) 虚拟元空间定义为

$$V_h(K) = \left\{ v \in H^1(K) : \Delta v \in \mathcal{P}_1(K), v|_e \in \mathcal{P}_1(e) \forall e \subset \partial K \right\}, \quad (3.7)$$

其自由度就是每个顶点上的函数值。单元上的双线性型被拆成两部分：

$$a_h(u_h, v_h) = \underbrace{a(\Pi^\nabla u_h, \Pi^\nabla v_h)}_{A_{\text{proj}}} + \underbrace{S_K((I - \Pi)u_h, (I - \Pi)v_h)}_{S_{\text{stab}}}. \quad (3.8)$$

3.4.1 投影算子

取缩放单项式基 $m_0 = 1, m_{k+1} = x_k - x_{K,k}$ (x_K 为单元形心)。投影 $\Pi^\nabla : V_h(K) \rightarrow \mathcal{P}_1(K)$ 满足

$$\int_K \nabla \Pi^\nabla v \cdot \nabla q \, dx = \int_K \nabla v \cdot \nabla q \, dx = - \int_K v \Delta q \, dx + \int_{\partial K} v (\nabla q \cdot n) \, ds = \int_{\partial K} v (\nabla q \cdot n) \, ds, \quad (3.9)$$

最后一步用到 $\Delta q = 0$ (q 是一次多项式)。由于 $v|_e$ 在每条边上是一次函数， $\int_e v (\nabla q \cdot n) \, ds$ 用梯形法则即精确：

$$\int_e v (\nabla q \cdot n) \, ds = \frac{|e|}{2} \left(v(a) (\nabla q \cdot n) + v(b) (\nabla q \cdot n) \right). \quad (3.10)$$

写成矩阵形式：设 $G = [\nabla m_0, \dots, \nabla m_d] \in \mathbb{R}^{d \times (d+1)}$, $M = |K| G^\top G$, $R_{\alpha i} = \int_{\partial K} \varphi_i (\nabla m_\alpha \cdot n) \, ds$, 则

$$\Pi = \text{pinv}(M) R \in \mathbb{R}^{(d+1) \times n}. \quad (3.11)$$

注意

常函数模式的规范化至关重要。若取 $\Pi_{0i} = 1/n$ (顶点值的算术平均), 则只有当单元“顶点算术平均 = 形心”时 (三角形、正方形成立, 一般多边形不成立) 投影才精确再生一次多项式。正确的规范化由 $p(x_K) = \bar{d} - \nabla p \cdot (\bar{X} - x_K)$ 给出:

$$\Pi_0 = \frac{1}{n} \mathbf{1}^\top - (\bar{X} - x_K) \Pi_{1:d}, \quad \bar{X} = \frac{1}{n} \sum_i X_i. \quad (3.12)$$

若忽略这一点, 稳定化项会惩罚线性函数, 相容性被破坏, 在不规则多边形网格上 L^2 收敛阶会从 2 掉到 0.5 以下。细节见第 11.1 节。

3.4.2 稳定化项

MosicaFE 默认采用对称半正定的稳定化

$$S_{\text{stab}} = \sigma (I - \Pi_{\text{dof}})^\top (I - \Pi_{\text{dof}}), \quad \Pi_{\text{dof}} = D \Pi, \quad D_i = [1, X_i - x_K], \quad (3.13)$$

其中缩放因子取

$$\sigma = \frac{\text{tr}(A_{\text{proj}})}{\text{tr}((I - \Pi_{\text{dof}})^\top (I - \Pi_{\text{dof}}))} \quad (3.14)$$

使两部分量级相当 (也自动对齐量纲)。传 `stabilization="legacy"` 可以复现早期脚本中 $(I - \Pi_{\text{dof}})/n$ 的形式 (非对称, 不推荐); `stabilization="none"` 只用于诊断, 因为纯投影项是奇异的。

3.4.3 载荷向量与质量矩阵

虚拟基函数没有显式表达式, 因此 $\int_K f \varphi_i$ 需要用扇形三角剖分 (3D 为四面体剖分) 上的分片线性插值近似。把单元从形心剖成 $T_1, \dots, T_n$, 每个 T_i 的三个角分别为形心 c 与 V_i, V_{i+1} , 其重心坐标为 L_0, L_1, L_2 , 则

$$\int_K f \varphi_j dx \approx c_0[j] \sum_i \int_{T_i} f L_0 dx + \int_{T_j} f L_1 dx + \int_{T_{j-1}} f L_2 dx, \quad (3.15)$$

其中 $c_0[j]$ 是基函数 φ_j 在形心处的插值 (取线性最小二乘重构的值, 既保证线性再生又保证单位分解)。对三角形网格该公式是精确的; 对一般多边形它保留了正确的几何权重。

注意

早期脚本对所有顶点统一用 $|K|/n$ 作为 $\int_K \varphi_i$ 。在四边形和三角形上这个近似恰好足够好, 但在不规则多边形上会严重高估“短边”上的顶点, 导致收敛停滞。这是我们把载荷与质量矩阵改为公式 (3.15) 的直接原因。

3.5 RT0×P0 混合离散

混合形式把 Poisson 方程写成一阶系统

$$q = -\nabla p, \quad \nabla \cdot q = f \text{ in } \Omega, \quad p = 0 \text{ on } \partial\Omega. \quad (3.16)$$

弱形式：求 $(q, p) \in RT0 \times P0$ 使得

$$\int_{\Omega} \nu \cdot q \, dx - \int_{\Omega} p \nabla \cdot \nu \, dx = 0, \quad \int_{\Omega} r \nabla \cdot q \, dx = \int_{\Omega} f r \, dx \quad (3.17)$$

对任意 (ν, r) 成立。离散后得到鞍点系统

$$\begin{pmatrix} M & B^T \\ B & 0 \end{pmatrix} \begin{pmatrix} \mathbf{Q} \\ \mathbf{P} \end{pmatrix} = \begin{pmatrix} 0 \\ \mathbf{f} \end{pmatrix}, \quad (3.18)$$

它是对称不定的，因此不能使用共轭梯度法。

表 3.2: RT0×P0 的自由度和边界条件

量	自由度位置	个数
通量 q_h	每个单元面（2D 边 / 3D 三角面），相邻单元共享	#facets
压力 p_h	每个单元一个常数	#cells
Dirichlet 条件 $p = 0$ 混合格式的自然边界条件，无需消去		

3.6 误差估计

定理 3.1 (Cea 引理). 设 u 是精确解、 u_h 是有限元解，则 $\|u - u_h\|_V \leq C \inf_{v_h \in V_h} \|u - v_h\|_V$ 。

定理 3.2 (先验误差估计). 若 $u \in H^{k+1}(\Omega)$ 、采用 k 次多项式，则

$$\|u - u_h\|_{L^2} \leq Ch^{k+1} |u|_{H^{k+1}}, \quad |u - u_h|_{H^1} \leq Ch^k |u|_{H^{k+1}}. \quad (3.19)$$

例 3.1 (本库实测的收敛阶). • P1 (三角形): L^2 为 $O(h^2)$, H^1 为 $O(h)$;

- P2 / Q2: L^2 为 $O(h^3)$, H^1 为 $O(h^2)$;
- VEM ($k = 1$, 任意多边形): L^2 为 $O(h^2)$, H^1 为 $O(h)$;
- RT0×P0: 压力 L^2 为 $O(h)$, 通量 L^2 为 $O(h)$ 。

第八章给出实测数据。

第四章 核心模块：网格与求积

4.1 网格数据结构

4.1.1 网格的组成部分

Mesh 对象保存以下信息：

表 4.1: Mesh 的主要属性

属性	类型	含义
<code>nodes</code>	<code>(n_nodes, dim)</code> 数组	顶点坐标
<code>elements</code>	<code>list[ndarray]</code>	每个单元的顶点编号，变长
<code>element_type</code>	<code>str</code>	<code>interval/triangle/quad/tet/hex/polygon/polyhedron</code>
<code>boundary_nodes</code>	<code>ndarray</code>	边界顶点编号（由 facet 拓扑自动判定）
<code>dim</code>	<code>int</code>	空间维数

要点

`elements` 故意设计成变长列表：三角形、四边形、五边形、Voronoi 多边形、四面体、六面体因此可以共用同一套组装循环。如果所有单元顶点数相同，可以通过 `mesh.uniform_vertices` 取出 `(n_cells, k)` 的数组视图。

4.1.2 facet 拓扑

余维 1 的单元面（facet）是边界判定的基础：1D 的 facet 是端点、2D 是边、3D 是三角形或四边形面。`mesh.facets` 给出全部 facet，`mesh.boundary_facets` 给出只被一个单元拥有的 facet，它们的顶点集合就是 Dirichlet 边界所在的位置。

性质 4.1. 对二维任意多边形网格，facet 就是多边形的边；对六面体网格按固定的 6 个面表展开；对一般三维多面体，可以通过 `Mesh(..., faces=...)` 显式提供面表。

注意

上面这套 `mesh.facets` 是”边界判定用的顶点集合”，它足以支撑 Lagrange 与虚拟元（自由度在顶点上）。但对 $H(\text{div})$ 协调的混合元（RT0）来说，还额外需要”哪些单元共享同一个面”以及”这个面的全局法向指向谁”，因此本库另有一层面拓扑：`core/facets.py`。下一小节给出它的用法。

4.1.3 面拓扑与定向：`core/facets.py`

RT0 的自由度是面法向通量 $\Phi_f = \int_f q \cdot n_f ds$ ，相邻两个单元必须共享同一个未知量。这要求显式地给出面的全局编号与定向。本库把它做成 `core` 层的一等公民 `FacetTopology` (`mesh.facet_topology` 惰性构建并缓存)：

表 4.2: `FacetTopology` 的主要属性 (`t = mesh.facet_topology`)

属性 / 方法	含义
<code>t.facets</code>	(<code>n_facets</code> , <code>d</code>): 每个面的升序顶点表（全局唯一键）
<code>t.cell_facets</code>	(<code>n_cells</code> , <code>d+1</code>): [<code>c</code> , <code>i</code>] 是单元 <code>c</code> 第 <code>i</code> 个局部面的全局编号
<code>t.signs</code>	± 1 : 全局法向是否指向该单元外部
<code>t.normals</code>	未归一化的全局法向（模长 = 面测度）
<code>t.counts</code>	相邻单元个数：内部面 2、边界面 1
<code>t.boundary_facets()</code>	边界面编号数组（ <code>t.n_boundary</code> 为个数）

```

1 from MosicaFE import Mesh
2
3 mesh = Mesh.box(nx=2, ny=2, nz=2, element_type="tet")
4 t = mesh.facet_topology
5
6 print(t.summary())           # cells=48, facets=120, interior=72, boundary=48
7 print(t.cell_facets[0])      # 单元 0 的 4 个局部面的全局编号
8 print(t.signs[0])           # 外法向符号 (+1 / -1)
9
10 # 内部面的两侧符号一定相反 —— 这就是  $H(\text{div})$  协调性的离散体现
11 c0, c1 = t.facet_cells[0]
12 print(t.signs[c0][list(t.cell_facets[c0]).index(0)] *
13       t.signs[c1][list(t.cell_facets[c1]).index(0)]) # -1

```

Listing 4.1: 示例 4.2: 面拓扑与定向

要点

`build_facet_topology` 会拒绝两类“会静默算错”的输入：非单纯形网格（四边形 / 多边形 / 六面体），以及存在被 3 个以上单元共用的面（不协调剖分）。这是 RT0 只接受三角形 / 四面体网格的技术原因。

4.2 网格生成

所有网格都由 `Mesh` 的类方法生成，且统一采用 逆时针（2D）/ 正体积（3D）取向。

```

1 from MosicaFE import Mesh
2
3 m1 = Mesh.interval(a=0.0, b=1.0, nx=20)           # 1D interval
4
5 m2 = Mesh.rectangle(nx=8, ny=8)                   # triangles
6 m3 = Mesh.rectangle(nx=8, ny=8, element_type="quad") # quadrilaterals
7
8 m4 = Mesh.box(nx=4, ny=4, nz=4)                   # tets (Kuhn split)
9 m5 = Mesh.box(nx=4, ny=4, nz=4, element_type="hex") # hexahedra
10 m6 = Mesh.box(nx=3, ny=3, nz=3, pattern="center") # 12 tets per cube
11
12 m7 = Mesh.pentagon(nx=8, ny=8)                   # pentagon mesh
13 m8 = Mesh.voronoi_polygon(n_points=100, seed=42) # Voronoi polygons
14
15 for m in (m1, m2, m3, m4, m5, m7, m8):
16     print(m.summary())

```

Listing 4.2: 示例 4.1: 生成各类网格

注 4.1 (Kuhn 剖分). `Mesh.box(..., element_type="tet")` 默认把每个小立方体剖成 6 个共享主对角线的四面体（Kuhn 剖分）。这样相邻立方体在公共面上的三角剖分方向一致，网格是协调的。`pattern="center"` 则额外插入立方体中心点，剖成 12 个四面体——与原始 VEM 三维脚本中的网格一致。

注 4.2 (Voronoi 网格). Voronoi 网格生成时在区域外做 8 方向镜像反射点，保证边界处的 Voronoi 单元闭合，然后裁剪回区域。裁剪与排序之后会自动去掉相邻重合顶点，避免出现退化单元（一个含重复顶点的多边形会让扇形剖分出现零面积三角形）。

4.3 网格属性与几何量

```

1 mesh = Mesh.rectangle(nx=8, ny=8)
2
3 print(mesh.dim, mesh.n_nodes, mesh.n_cells)    # 2 81 128

```

表 4.3: 网格工厂速查

工厂	区域	单元
<code>Mesh.interval(a,b,nx)</code>	$[a, b]$	线段
<code>Mesh.rectangle(..., "triangle")</code>	矩形	三角形
<code>Mesh.rectangle(..., "quad")</code>	矩形	四边形
<code>Mesh.box(..., "tet")</code>	长方体	四面体
<code>Mesh.box(..., "hex")</code>	长方体	六面体
<code>Mesh.pentagon(nx, ny)</code>	单位正方形	五边形
<code>Mesh.voronoi_polygon(...)</code>	单位正方形	Voronoi 多边形
<code>Mesh.from_arrays(...)</code>	自定义	自动推断

```

4 print(mesh.get_mesh_size())           # max cell diameter
5 print(mesh.cell_measures())           # array of areas
6
7 v = mesh.get_cell_vertices(0)         # (k, dim) coordinates
8 h = mesh.get_cell_diameter(0)
9 c = mesh.get_cell_centroid(0)
10 print(mesh.boundary_nodes[:5])

```

Listing 4.3: 示例 4.2: 读取网格信息

4.4 网格加密

```

1 coarse = Mesh.rectangle(nx=4, ny=4)
2 fine = coarse.refine_uniform()
3 print(coarse.n_cells, fine.n_cells)   # 32 128

```

Listing 4.4: 示例 4.3: 均匀加密

`refine_uniform()` 只对结构化网格 (`interval` / `rectangle` / `box` / `pentagon`) 有效: 它把各方向的网格数翻倍后重新生成, 因此三角形网格的单元数变为 4 倍、六面体网格变为 8 倍。对 Voronoi 这类非结构网格会抛出 `ValueError`。

4.5 参考单元

单纯形使用仿射映射 $x = v_0 + J_K \hat{x}$; 四边形与六面体使用 $[0, 1]^d$ 上的双线性 / 三线性映射, 其雅可比矩阵依赖参考点 $\hat{x}$, 因此需要逐求积点计算。

表 4.4: 参考单元约定

单元类型	参考单元
interval	$[0, 1]$
triangle	$\{\xi \geq 0, \eta \geq 0, \xi + \eta \leq 1\}$
quad	$[0, 1]^2$, 顶点顺序 $(0, 0), (1, 0), (1, 1), (0, 1)$
tet	标准四面体, 顶点 $(0, 0, 0), (1, 0, 0), (0, 1, 0), (0, 0, 1)$
hex	$[0, 1]^3$, 先 $z = 0$ 面逆时针 4 点, 再 $z = 1$ 面对应 4 点

```

1 import numpy as np
2 from MosicaFE.core.utils import (
3     compute_jacobian, transform_gradient, apply_affine_map,
4     polygon_area, polyhedron_volume,
5 )
6
7 quad = np.array([[0,0],[2,0],[2,2],[0,2]], float)
8 J, detJ = compute_jacobian(quad, np.array([0.5, 0.5]))
9 print(J, detJ) # [[2, 0], [0, 2]] 4.0
10 print(polygon_area(quad)) # 4.0
11 print(apply_affine_map([[0.5,0.5]], quad)) # [[1, 1]]

```

Listing 4.5: 示例 4.4: 几何工具

注意

雅可比约定。`compute_jacobian` 对两类单元返回同一个约定的雅可比：列是物理坐标对参考坐标的偏导，

$$J_{ab} = \frac{\partial x_a}{\partial \xi_b}, \quad J = \begin{bmatrix} \partial x / \partial \xi_1 & \cdots & \partial x / \partial \xi_d \end{bmatrix}, \quad (4.1)$$

只有这样，梯度变换 `transform_gradient(dN, J)`（内部用 $J^{-\top}$ ）与四边形 / 六面体的 Newton 逆映射（内部用 J^{-1} ）才是对的。自己扩展代码时不要写成 $dN^{\top} V$ ——那是转置雅可比。对轴对齐的矩形 / 长方体， J 是对角阵、转置不变，这个错误会被完全掩盖；而网格一旦旋转或畸变（ $J \neq J^{\top}$ ），梯度、刚度矩阵与插值就会静默算错。本库在 2026-09-21 修掉过这一处（第 7 号 bug），回归测试见 `tests/test_tensor_geometry.py`：它用有限差分核对 J ，并检验旋转 $0^\circ/15^\circ/30^\circ/45^\circ$ 时误差逐位相同。

4.6 求积规则

```

1 from MosicaFE.core.quadrature import QuadratureRule, cell_quadrature

```

```
2
3 quad = QuadratureRule.get_quadrature("triangle", order=3)
4 print(quad.n_points, quad.weights.sum())      # 4 0.5
5
6 f = lambda xi: xi[0]**2 + xi[1]**2
7 value = sum(quad.weights[q] * f(quad.points[q]) for q in range(quad.n_points))
8 print(value)                                  # 1/6
9
10 # 映射到物理单元（自动处理多边形 / 多面体的扇形剖分）
11 mesh = Mesh.voronoi_polygon(n_points=20, seed=1)
12 pts, w = cell_quadrature(mesh, 0, order=3)
13 print(w.sum(), mesh.get_cell_measure(0))
```

Listing 4.6: 示例 4.5: 使用求积规则

表 4.5: 可用的求积规则（order 为需精确积分的多项式次数）

单元	规则	说明
interval	Gauss-Legendre	$n = \lceil (order + 1)/2 \rceil$ 点, $\int_0^1$ 归一
triangle	重心 / 3 点 / 4 点 / 6 点	分别精确到 1/2/3/4 次
quad	张量 Gauss	每个方向 n 点
tet	1 点 / 4 点 / 5 点	精确到 1/2/3 次
hex	张量 Gauss	每个方向 n 点

注意

张量积单元的求积阶需要按**每个方向**的次数来理解。Q1 单元刚度矩阵的被积函数在每个方向上是 2 次，所以每个方向至少需要 2 个 Gauss 点——MosicaFE 内部会自动把阶数取成 $\max(2, 2*p)$ ，用户一般不必手工指定。早期版本在这里少取了一个点，导致三维六面体的收敛阶算出来是负数。

第五章 离散空间

5.1 空间抽象

所有空间都继承 `BaseFESpace`，并向上提供矩阵级接口：

表 5.1: `BaseFESpace` 的关键方法

方法	含义
<code>stiffness_matrix()</code>	$A_{ij} = \int \nabla \varphi_j \cdot \nabla \varphi_i$
<code>mass_matrix()</code>	$M_{ij} = \int \varphi_j \varphi_i$
<code>load_vector(f)</code>	$b_i = \int f \varphi_i$
<code>boundary_dofs()</code>	Dirichlet 自由度编号
<code>interpolate(f)</code>	把函数插值到自由度上
<code>evaluate(cell,u,pts)</code>	在单元内把自由度重构成函数并求值
<code>evaluate_gradient(cell,u,pts)</code>	同上，返回梯度
<code>nodal_values(u)</code>	把自由度向量还原到网格节点（可视化用）
<code>get_cell_dofs(i)</code>	第 i 个单元的全局自由度编号
<code>get_dof_coordinates(k)</code>	第 k 个自由度的物理坐标
<code>eval_basis(xi) /</code> <code>eval_basis_grad(xi)</code>	参考单元上的基函数（仅显式基函数空间）

要点

这组接口是“新增 PDE 只需十几行”的根本原因：物理层永远只做矩阵的线性组合与边界处理，不必关心单元循环。

5.2 Lagrange 元（经典有限元）

5.2.1 P1 三角形

参考三角形上，用重心坐标 $\lambda_0 = 1 - \xi - \eta$ ， $\lambda_1 = \xi$ ， $\lambda_2 = \eta$ ：

$$\hat{\varphi}_0 = 1 - \xi - \eta, \quad \hat{\varphi}_1 = \xi, \quad \hat{\varphi}_2 = \eta. \quad (5.1)$$

5.2.2 P2 三角形

$$\begin{aligned} \hat{\varphi}_0 &= \lambda_0(2\lambda_0 - 1), & \hat{\varphi}_1 &= \lambda_1(2\lambda_1 - 1), & \hat{\varphi}_2 &= \lambda_2(2\lambda_2 - 1), \\ \hat{\varphi}_3 &= 4\lambda_0\lambda_1 \text{ (棱0-1)}, & \hat{\varphi}_4 &= 4\lambda_1\lambda_2 \text{ (棱1-2)}, & \hat{\varphi}_5 &= 4\lambda_2\lambda_0 \text{ (棱2-0)}. \end{aligned} \quad (5.2)$$

5.2.3 张量积元

四边形与六面体使用 1D Lagrange 基的张量积。以 Q2 为例，1D 节点取 $\{0, \frac{1}{2}, 1\}$ ：

$$N_0(t) = 2t^2 - 3t + 1, \quad N_1(t) = -4t^2 + 4t, \quad N_2(t) = 2t^2 - t, \quad (5.3)$$

单元基函数为 $N_{ijk}(\xi, \eta, \zeta) = N_i(\xi)N_j(\eta)N_k(\zeta)$ 。由于节点集是完整张量网格， $\{N_{ijk}\}$ 自动构成对偶于节点值的节点基。

5.2.4 自由度的全局编号

MosicaFE 用一个”登记簿”把几何实体映射到全局自由度：

- 顶点自由度：以全局顶点编号为 key；
- 棱自由度：以两个端点的有序对为 key；
- 面自由度（3D）：以面的 4 个顶点有序元组为 key；
- 单元内部自由度：以单元编号为 key。

于是相邻单元自动共享棱 / 面自由度，全局空间是 H^1 协调的。

```

1 from MosicaFE import Mesh, FESpace
2
3 mesh = Mesh.rectangle(nx=8, ny=8)
4
5 V1 = FESpace.lagrange(mesh, degree=1)
6 V2 = FESpace.lagrange(mesh, degree=2)
7 print(V1.n_dofs, V2.n_dofs)      # 81 289
8
9 print(V2.get_cell_dofs(0))      # local -> global DOF map
10 print(V2.get_dof_coordinates(0))
11 print(V2.boundary_dofs()[:6])

```

Listing 5.1: 示例 5.1: 创建空间并查看自由度

5.3 虚拟元空间

```

1 from MosicaFE import Mesh, FESpace
2
3 mesh = Mesh.voronoi_polygon(n_points=100, seed=0)
4 V = FESpace.vem(mesh)                # k = 1, vertex DOFs only
5
6 print(V.n_dofs, mesh.n_nodes)        # equal
7 K = V.stiffness_matrix()             # projection + stabilization
8 M = V.mass_matrix()
9 b = V.load_vector(lambda x: 1.0)
10
11 print("row sums of M equal b?",
12       abs(b - M.sum(axis=1)).max() < 1e-12)

```

Listing 5.2: 示例 5.2: 虚拟元空间（任意多边形）

性质 5.1. VEMSpace 的刚度矩阵满足

- (1) 对称（因为稳定化项写成 Gram 矩阵的形式）；
- (2) 半正定，核空间中包含常函数；
- (3) 一次多项式被精确再生（单元矩阵作用在任意线性自由度向量上，给出的能量等于 $\int_K |\nabla p|^2$ ）。

第 (3) 条就是“patch test”，第 11.1 节会说明它有多重要。

表 5.2: VEMSpace 的 stabilization 选项

取值	含义
"trace"（默认）	$\sigma(I - \Pi_{\text{dof}})^\top (I - \Pi_{\text{dof}})$ ，对称半正定， σ 由迹比确定。推荐使用
"legacy"	早期脚本的 $(I - \Pi_{\text{dof}})/n$ ，非对称，在不规则多边形上会发散，仅用于复现旧结果
"none"	只用投影项 A_{proj} 。单元矩阵是奇异的，仅用于诊断，不要用于求解

5.4 RT0×P0 混合空间

```

1 from MosicaFE import Mesh, FESpace
2
3 mesh = Mesh.rectangle(nx=16, ny=16)    # triangle mesh

```

```

4 V = FESpace.rt0(mesh)
5
6 print(V.n_flux, V.n_pressure, V.n_dofs)    # facets, cells, sum
7 print(V.cell_flux_dofs(0))                 # local facet -> global flux DOF
8 print(V.facet_signs(0))                   # outward orientation signs
9 print(V.local_flux_mass(0))                # 单元 0 的精确  $RT_0$  质量矩阵 ( $3 \times 3$ )

```

Listing 5.3: 示例 5.3: 混合空间

5.4.1 空间层给出的两个块

RT_0 空间与 `LagrangeSpace` / `VEMSpace` 的接口是平行的，只是把”单个椭圆算子”换成了”鞍点问题的两个块”：

表 5.3: $RT_0 \times P_0$ 的矩阵原语（供物理层组合）

方法	形状	含义
<code>V.flux_mass_matrix()</code>	(Φ, Φ)	$M_{ij} = \int_K \varphi_i \cdot \varphi_j$, 精确闭式, 无需求积
<code>V.divergence_matrix()</code>	(P, Φ)	$B_{K,i} = \int_K \nabla \cdot \varphi_i = \sigma_{K,i}$, 恒为 ± 1
<code>V.pressure_load_vector(f)</code>	$(P,)$	$\int_K f$
<code>V.interpolate_pressure(f)</code>	$(P,)$	f 的单元平均 (L^2 投影到 P_0)
<code>V.interpolate_flux(q)</code>	$(\Phi,)$	$\Phi_f = \int_f q \cdot n_f ds$
<code>V.evaluate_flux(c,Q,pts)</code>	(n_q, d)	单元内重构 $q_h = \sum_i F_i \varphi_i$
<code>V.locate_cell(x)</code>	int	用重心坐标定位点所在单元

```

1 from MosicaFE import Mesh, FESpace
2 from MosicaFE._compat import bmat
3
4 mesh = Mesh.rectangle(nx=16, ny=16)
5 V = FESpace.rt0(mesh)
6
7 M = V.flux_mass_matrix()           # (Phi, Phi)
8 B = V.divergence_matrix()          # (P, Phi)
9 f = V.pressure_load_vector(src)    # (P, )
10
11 A = bmat([[M, -B.T], [B, None]])  # [[M, -B^T], [B, 0]]

```

Listing 5.4: 示例 5.4: 直接使用两个块

注意

RT0 空间不提供 `stiffness_matrix()` / `mass_matrix()` / `load_vector()`，也不提供 `interpolate()`：它对应的是鞍点问题，而且同时携带两个场，没有”单一算子 / 单一插值”可言。调用这些方法会抛出 `NotImplementedError`，并在消息里指出该用哪个方法（例如 `mass_matrix()` → `flux_mass_matrix()`，`load_vector()` → `pressure_load_vector()`）。日常使用请直接配合 `MixedPoissonProblem`。

5.5 空间工厂 FESpace

```
1 from MosicaFE import Mesh, FESpace
2
3 mesh = Mesh.rectangle(nx=16, ny=16)
4
5 V_fem = FESpace.lagrange(mesh, degree=2)      # P2
6 V_vem = FESpace.vem(mesh, degree=1)          # VEM k=1
7 V_rt0 = FESpace.rt0(mesh)                    # RT0 x P0
8
9 # by name (used by solve_poisson(element=...))
10 print(FESpace.available())
11 V_any = FESpace.create(mesh, family="vem", degree=1)
```

Listing 5.5: 示例 5.4: 三种空间族

表 5.4: 字符串形式的”元”名字（`solve_poisson(element=...)` 使用）

名字	空间族	可用网格
"P1" / "P2"	Lagrange	三角形、四面体
"Q1" / "Q2"	Lagrange（张量积）	四边形、六面体
"VEM" / "VEM1"	虚拟元 $k = 1$	任意多边形 / 多面体
"RT0" / "RT0-P0" / "MIXED"	RT0×P0	三角形、四面体

第六章 物理问题与组装

6.1 问题接口

所有 PDE 都继承 BasePhysics, 只需实现一个方法:

```
1 class BasePhysics(ABC):
2     def __init__(self, space, f=None, g=None, **kwargs):
3         self.space = space
4         self.mesh = space.mesh
5         self.n_dofs = space.n_dofs
6         self.f = f or (lambda x: 0.0)      # source term
7         self.g = g or (lambda x: 0.0)      # Dirichlet data
8         self.boundary_dofs = space.boundary_dofs()
9
10    @abstractmethod
11    def assemble(self):
12        """return (A, b) with boundary conditions applied"""
13
14    def solve(self, method="auto", **kwargs):
15        A, b = self.assemble()
16        return Solver.solve(A, b, method=method, **kwargs)
```

Listing 6.1: 示例 6.1: BasePhysics 接口

6.2 Poisson 问题

```
1 @register_problem("poisson")
2 class PoissonProblem(BasePhysics):
3     def __init__(self, space, f=None, g=None, kappa=1.0, **kwargs):
4         super().__init__(space, f=f, g=g, **kwargs)
5         self.kappa = kappa
6
7     def assemble(self):
8         A = self.space.stiffness_matrix()
9         if float(self.kappa) != 1.0:
10             A = float(self.kappa) * A
```

```

11         b = self.space.load_vector(self.f)
12         A, b = self.apply_boundary_conditions(A, b)
13         return A, b

```

Listing 6.2: 示例 6.2: Poisson 问题的组装（完整实现）

要点

整个实现只有 8 行，而且它对 **Lagrange** 元与虚拟元同时成立。这正是把”单元循环”下沉到 `MosicaFE.spaces` 层之后的收益。

6.3 边界条件

6.3.1 强加 Dirichlet 条件的正确做法

对边界自由度 j 强加 $u_j = g_j$ ，标准做法是”消行消列 + 右端项修正”：

(1) 把边界列对内部行的贡献搬到右端项：

$$b_i \leftarrow b_i - \sum_{j \in \mathcal{D}} A_{ij} g_j, \quad \forall i \notin \mathcal{D}; \quad (6.1)$$

(2) 删去所有涉及 $\mathcal{D}$ 的行与列；

(3) 对 $j \in \mathcal{D}$ 置 $A_{jj} = 1$, $b_j = g_j$ 。

注意

第 (1) 步在齐次条件 $g = 0$ 时”看起来”可以省略，因此很多演示程序都漏掉了它；一旦边界数据非零，解就是错的。MosicaFE 用一个简单而灵敏的测试——非齐次线性 **patch test**——来防止这类错误回归：

$$u(x) = 1 + 2x_1 - \frac{1}{2}x_2, \quad f \equiv 0, \quad g = u|_{\partial\Omega}, \quad (6.2)$$

此时精确解属于离散空间，数值解应当精确（误差 $\sim 10^{-15}$ ）。修复前的实测误差是 2.875。

6.3.2 其它边界条件

- **Neumann 条件**：向 `load_vector` 的结果中加入 $\int_{\Gamma_N} h \varphi_i ds$ 即可，需要边界 facet 的求积规则；
- **RT0 混合格式**： $p = 0$ 是自然边界条件，无需消去任何自由度（这也是混合格式在流量问题上更”自然”的原因）；
- 非齐次 $p = g \neq 0$ 在混合格式下需要额外处理边界通量项，当前版本会显式报错而不是给出错误结果。

6.4 组装算法

Algorithm 1 全局组装 (MosicaFE 的 `BaseFESpace._assemble`)

Require: 空间 V 、单元局部矩阵生成函数 `local_matrix(cell)`

```

1: 初始化三元组列表 rows, cols, vals
2: for 每个单元  $c = 0, \dots, n_{\text{cells}} - 1$  do
3:    $\mathcal{D}_c \leftarrow \text{get\_cell\_dofs}(c)$ 
4:    $K_c \leftarrow \text{local\_matrix}(c)$ 
5:   把  $\mathcal{D}_c$  的行 / 列索引与  $K_c$  展平后追加到三元组列表
6: end for
7: 由 COO 三元组构造矩阵 (重复下标自动求和)
8: if 要求对称 then
9:    $A \leftarrow \frac{1}{2}(A + A^\top)$ 
10: end if
11: return  $A$ 

```

注意

右端项组装必须使用”逐项累加”语义 (numpy 的 `np.add.at`)。若直接用 `b[dofs] += local`, 当 `dofs` 中出现重复下标时 (例如网格里存在重复顶点), numpy 的缓冲赋值会丢掉累加, 而稀疏矩阵的 COO 组装却会正确求和——两者不一致会造成难以察觉的错误。

6.5 使用 Poisson 问题

```

1 import numpy as np
2 from MosicaFE import Mesh, FESpace, PoissonProblem
3
4 def source(x):
5     return 2*np.pi**2 * np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])
6
7 mesh = Mesh.rectangle(nx=32, ny=32)
8 V = FESpace.lagrange(mesh, degree=2)
9
10 problem = PoissonProblem(V, f=source, g=lambda x: 0.0, kappa=1.0)
11
12 A, b = problem.assemble()
13 print("system size:", A.shape, "non-zeros:", A.nnz)
14
15 u = problem.solve(method="direct")
16 print("solution size:", u.size)

```

Listing 6.3: 示例 6.3: 组装与求解

6.6 PDE 注册表

```
1 from MosicaFE import FESpace, Mesh, available_problems, create_problem, run
2
3 print(available_problems())          # ['mixed_poisson', 'poisson', ...]
4
5 mesh = Mesh.rectangle(nx=32, ny=32)
6 problem = create_problem("poisson", FESpace.vem(mesh),
7                           f=source, g=lambda x: 0.0)
8 u = problem.solve()
9
10 u2 = run("poisson", mesh, element="VEM", f=source, g=lambda x: 0.0)
```

Listing 6.4: 示例 6.4: 按名字创建与求解方程

第七章 线性求解器

7.1 求解器接口

```
1 import numpy as np, time
2 from MosicaFE import FESpace, Mesh, PoissonProblem, Solver
3
4 mesh = Mesh.rectangle(nx=32, ny=32)
5 problem = PoissonProblem(FESpace.lagrange(mesh, 2), f=source,
6                           g=lambda x: 0.0)
7 A, b = problem.assemble()
8
9 t0 = time.time(); u_direct = Solver.direct(A, b); t1 = time.time()
10 u_cg = Solver.conjugate_gradient(A, b, tol=1e-10)
11 u_gmres = Solver.gmres(A, b, tol=1e-10, restart=30)
12 u_bicg = Solver.bicgstab(A, b, tol=1e-10)
13
14 print("direct %.3f s, diff vs CG: %.3e"
15       % (t1 - t0, np.linalg.norm(u_direct - u_cg)))
```

Listing 7.1: 示例 7.1: Solver 接口

也可以让问题对象自己选择:

```
1 u = problem.solve(method="auto") # auto/direct/cg/gmres/bicgstab/minres
```

7.2 直接法

- 稀疏矩阵: `scipy.sparse.linalg.spsolve` (内部为稀疏 LU / Cholesky);
- 稠密矩阵: `numpy.linalg.solve`;
- 复杂度: 2D 约 $O(n^{1.5})$, 3D 约 $O(n^2)$ 。

表 7.1: 可用的 Krylov 求解器

方法	适用矩阵	备注
conjugate_gradient	对称正定	每步 $O(\text{nnz})$, 迭代数 $O(\sqrt{\kappa})$
gmres	一般 (非对称)	带重启, 存储 $O(\text{restart} \times n)$
bicgstab	一般	每步两次矩阵向量积, 内存小
minres	对称不定	例如 RT0 的鞍点系统

7.3 迭代法

注意

共轭梯度法要求矩阵对称正定。虚拟元的刚度矩阵在默认稳定化下是对称的，但如果选了 `stabilization="legacy"`，矩阵不再对称，CG 会失效（甚至不报错地给出错误结果）。鞍点系统请用直接法或 MINRES。

7.4 求解器选择指南

表 7.2: 求解器选择建议

问题规模	矩阵类型	推荐求解器	备注
$< 2 \times 10^4$ 自由度	任意	直接法	精确、无需调参
$> 2 \times 10^4$	对称正定	CG	建议配预处理
$> 2 \times 10^4$	对称不定	MINRES 或直接法	鞍点问题
$> 2 \times 10^4$	非对称	GMRES / BiCGSTAB	需要重启参数

```
1 from MosicaFE import Solver
2
3 print(Solver.recommend(1000, symmetric=True))      # 'direct'
4 print(Solver.recommend(10**6, symmetric=True))     # 'cg'
5 print(Solver.recommend(10**6, symmetric=False))    # 'gmres'
```

Listing 7.2: 示例 7.2: 自动选择

第八章 误差分析与收敛性

8.1 误差范数

$$\|e\|_{L^2} = \left(\int_{\Omega} |u - u_h|^2 dx \right)^{1/2}, \quad \|e\|_{L^2}^2 \approx \sum_K \sum_q w_q |u(x_q) - u_h(x_q)|^2 |\det J_K|, \quad (8.1)$$

$$|e|_{H^1} = \left(\int_{\Omega} |\nabla u - \nabla u_h|^2 dx \right)^{1/2}, \quad \|e\|_{L^\infty} = \max_{x \in \Omega} |u(x) - u_h(x)|. \quad (8.2)$$

```
1 import numpy as np
2
3 def exact(x):
4     return np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])
5
6 def exact_grad(x):
7     return np.array([np.pi*np.cos(np.pi*x[0])*np.sin(np.pi*x[1]),
8                      np.pi*np.sin(np.pi*x[0])*np.cos(np.pi*x[1])])
9
10 u = problem.solve()
11 l2 = problem.compute_error(u, exact, "L2")
12 linf = problem.compute_error(u, exact, "Linf")
13 h1 = problem.compute_error(u, exact, "H1", exact_grad=exact_grad)
14
15 print("%.6e %.6e %.6e" % (l2, h1, linf))
16
17 norms = problem.compute_errors(u, exact, exact_grad)
```

Listing 8.1: 示例 8.1: 计算三种范数

注 8.1. 虚拟元的基函数没有显式表达式，`VEMSpace.evaluate` 用的是“顶点自由度上的线性最小二乘重构”。该重构误差与 u_h 本身同阶，因此实测收敛阶与 u_h 的真实收敛阶一致（见第 8.4 节）。

8.2 收敛性研究框架

```

1 from MosicaFE import FESpace, Mesh, PoissonProblem, convergence_study
2 from MosicaFE.postprocess import estimate_convergence_rate
3
4 def mesh_factory(level):
5     n = 4 * 2**level
6     return Mesh.rectangle(nx=n, ny=n)
7
8 def problem_factory(mesh):
9     V = FESpace.lagrange(mesh, degree=2)
10    return PoissonProblem(V, f=source, g=lambda x: 0.0)
11
12 h, errors = convergence_study(mesh_factory, problem_factory,
13                               exact, exact_grad,
14                               refinement_levels=5, plot=True,
15                               filename="conv_p2.png")
16
17 print(estimate_convergence_rate(h, errors["L2"], use_last=3)) # ~3.0

```

Listing 8.2: 示例 8.2: 自动收敛性研究

8.3 收敛阶估计

由两层级误差

$$\text{rate} = \frac{\log(e_1/e_2)}{\log(h_1/h_2)}. \quad (8.3)$$

更稳健的做法是对 $\log e - \log h$ 做最小二乘拟合 (`estimate_convergence_rate(h, errors)` 就是这样实现的), 并且可以用 `use_last=k` 只取最后 k 层以逼近渐近阶。

8.4 实测验证结果

复现方式:

```
& "D:\anaconda\envs\fealpy\python.exe" MosicaFE\tests\run_checks.py
```

表 8.1: 各方法的实测收敛阶（制造解，网格逐次加密）

元	网格	实测 L^2	实测 H^1	理论 (L^2/H^1)
P1	三角形	1.98	0.99	2 / 1
P2	三角形	3.00	1.99	3 / 2
Q1	四边形	2.00	1.00	2 / 1
Q2	四边形	3.00	2.00	3 / 2
VEM $k = 1$	三角形	1.98	0.99	2 / 1
VEM $k = 1$	四边形	1.99	1.00	2 / 1
VEM $k = 1$	五边形	2.00	1.00	2 / 1
VEM $k = 1$	Voronoi 多边形	1.97	1.01	2 / 1
P1	四面体	1.95	0.98	2 / 1
Q1	六面体	2.01	1.00	2 / 1
VEM $k = 1$	六面体	1.97	0.97	2 / 1
RT0×P0	三角形（压力）	1.00	—	1
RT0×P0	三角形（通量）	1.00	—	1

第九章 可视化

9.1 可视化工具一览

表 9.1: MosicaFE.postprocess 中的绘图函数

函数	用途
<code>plot_solution(u, mesh, V, ...)</code>	画解。1D 画曲线；2D 画等值线填充或三维曲面；3D 画 $z = \text{const}$ 切片。多边形网格自动从形心做扇形三角化
<code>plot_mesh(mesh, ...)</code>	画网格（3D 画中间切片）
<code>plot_error(u, mesh, space, exact, ...)</code>	画逐点误差 $ u_h - u $
<code>plot_convergence(h, errors, ...)</code>	log-log 收敛曲线，并叠加 $O(h^k)$ 参考斜率
<code>plot_pressure_2d(mesh, P, ...)</code>	RT0 的分片常数压力
<code>plot_flux_2d(mesh, Q, ...)</code>	通量箭头图
<code>compare_solutions(entries, ...)</code>	多组解并排比较

通用参数: `title=`、`filename=`（保存 PNG）、`show=`（弹窗）、`dpi=`、`ax=`（画到已有坐标轴上以便拼图）。

9.2 画解

```
1 from MosicaFE import Mesh, FESpace, PoissonProblem
2 from MosicaFE.postprocess import plot_solution, plot_mesh, plot_error
3
4 # --- 2D contour ---
5 mesh = Mesh.rectangle(nx=32, ny=32)
6 V = FESpace.lagrange(mesh, 2)
7 u = PoissonProblem(V, f=source, g=lambda x: 0.0).solve()
```

```

8 plot_solution(u, mesh, V, title="FEM P2", filename="sol2d.png")
9
10 # --- 2D surface ---
11 plot_solution(u, mesh, V, mode="surface", filename="sol_surface.png")
12
13 # --- polygon mesh + VEM solution ---
14 mp = Mesh.voronoi_polygon(n_points=200, seed=3)
15 Vv = FESpace.vem(mp)
16 uv = PoissonProblem(Vv, f=source, g=lambda x: 0.0).solve()
17 plot_solution(uv, mp, Vv, title="VEM on polygons",
18             filename="sol_vem.png")
19 plot_error(uv, mp, Vv, exact, filename="err_vem.png")
20
21 # --- 3D slice ---
22 m3 = Mesh.box(nx=8, ny=8, nz=8)
23 V3 = FESpace.lagrange(m3, 1)
24 u3 = PoissonProblem(V3, f=source3d, g=lambda x: 0.0).solve()
25 plot_solution(u3, m3, V3, z_slice=0.5, filename="sol3d_slice.png")
26 plot_mesh(m3, filename="mesh3d.png")

```

Listing 9.1: 示例 9.1: 2D 与 3D 解的绘制

9.3 画收敛曲线

```

1 from MosicaFE.postprocess import plot_convergence
2
3 plot_convergence(h, errors, filename="convergence.png",
4                 reference_rates=(1, 2, 3))

```

Listing 9.2: 示例 9.2: 收敛曲线

注意

所有绘图函数在没有安装 `matplotlib` 时只会打印提示并返回 `None`，不会抛异常，因此批量计算的脚本可以放心调用。

第十章 完整示例

10.1 示例一：两行求解 Poisson 方程

```
1 import numpy as np
2 from MosicaFE import Mesh, solve_poisson
3
4 source = lambda x: 2*np.pi**2 * np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])
5
6 u = solve_poisson(Mesh.rectangle(nx=32, ny=32), element="P2", f=source)
7 print(u.size)           # 4225
```

Listing 10.1: 示例 10.1: 一行式求解

10.2 示例二：换一种元（同一段代码，三种方法）

```
1 import numpy as np
2 from MosicaFE import FESpace, Mesh, MixedPoissonProblem, PoissonProblem
3
4 def exact(x):
5     return np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])
6
7 def grad(x):
8     return np.array([np.pi*np.cos(np.pi*x[0])*np.sin(np.pi*x[1]),
9                     np.pi*np.sin(np.pi*x[0])*np.cos(np.pi*x[1])])
10
11 def source(x):
12     return 2*np.pi**2 * exact(x)
13
14 def zero(x):
15     return 0.0
16
17 mesh = Mesh.rectangle(nx=32, ny=32)
18
19 # (a) classic FEM: P2 Lagrange
20 V = FESpace.lagrange(mesh, degree=2)
```

```

21 p = PoissonProblem(V, f=source, g=zero)
22 u = p.solve()
23 print("FEM P2 :", p.compute_errors(u, exact, grad))
24
25 # (b) VEM: only the space changes
26 V = FESpace.vem(mesh)
27 p = PoissonProblem(V, f=source, g=zero)
28 u = p.solve()
29 print("VEM k=1:", p.compute_errors(u, exact, grad))
30
31 # (c) RT0 x P0 mixed: space + problem class
32 V = FESpace.rt0(mesh)
33 p = MixedPoissonProblem(V, f=source)
34 Q, P = p.solve()
35 print("RT0-P0 :", p.compute_errors((Q, P), exact, lambda x: -grad(x)))

```

Listing 10.2: 示例 10.2: FEM / VEM / RT0 共用同一套流程

运行结果 (32×32 网格):

```

FEM P2 : {'L2': 6.87e-05, 'Linf': 2.41e-03, 'H1': 7.38e-03}
VEM k=1: {'L2': 1.35e-03, 'Linf': 3.18e-03, 'H1': 1.09e-01}
RT0-P0 : {'L2': 1.64e-02, 'Linf': 4.55e-02, 'flux_L2': 6.30e-02}

```

10.3 示例三：多边形网格上的虚拟元

```

1 import numpy as np
2 from MosicaFE import FESpace, Mesh, PoissonProblem, convergence_study
3 from MosicaFE.postprocess import estimate_convergence_rate, plot_solution
4
5 def mesh_factory(level):
6     return Mesh.voronoi_polygon(n_points=50 * 4**level, seed=7)
7
8 def problem_factory(mesh):
9     return PoissonProblem(FESpace.vem(mesh), f=source, g=lambda x: 0.0)
10
11 h, errors = convergence_study(mesh_factory, problem_factory,
12                               exact, grad, refinement_levels=3,
13                               verbose=True)
14 print("L2 rate:", estimate_convergence_rate(h, errors["L2"]))
15
16 mesh = Mesh.voronoi_polygon(n_points=200, seed=7)
17 V = FESpace.vem(mesh)
18 u = PoissonProblem(V, f=source, g=lambda x: 0.0).solve()

```

```

19 plot_solution(u, mesh, V, title="VEM on Voronoi mesh",
20               filename="vem_voronoi.png")

```

Listing 10.3: 示例 10.3: Voronoi 多边形网格 + 收敛性研究

注意

随机 Voronoi 网格的 h 用“最大单元直径”会很不稳定，建议改用平均单元直径作为横坐标，convergence_study 里传自定义 mesh_factory 即可（示例中用的是 mesh.get_mesh_size()，若要更稳健可在回调里替换）。

10.4 示例四：自定义新 PDE (Helmholtz)

```

1  import numpy as np
2  from MosicaFE import FESpace, Mesh
3  from MosicaFE.physics.base import BasePhysics
4  from MosicaFE.physics.registry import register_problem
5
6  @register_problem("helmholtz")
7  class HelmholtzProblem(BasePhysics):
8      """-Laplacian(u) - k^2 u = f, weak form: (grad u, grad v) - k^2 (u, v) = (f, v)"""
9
10     def __init__(self, space, f=None, g=None, k=1.0, **kwargs):
11         super().__init__(space, f=f, g=g, **kwargs)
12         self.k = k
13
14     def assemble(self):
15         A = self.space.stiffness_matrix() - self.k**2 * self.space.mass_matrix()
16         b = self.space.load_vector(self.f)
17         return self.apply_boundary_conditions(A, b)
18
19     # use it with any space family
20     k = 1.0
21     exact = lambda x: np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])
22     source = lambda x: (2*np.pi**2 - k**2) * exact(x)
23
24     mesh = Mesh.rectangle(nx=32, ny=32)
25     V = FESpace.vem(mesh) # works with VEM too
26     problem = HelmholtzProblem(V, f=source, g=lambda x: 0.0, k=k)
27     u = problem.solve(method="direct")
28     print("max error:", np.abs(u - V.interpolate(exact)).max())

```

Listing 10.4: 示例 10.4: 新增一门方程只需十几行

输出: max error: 9.310e-04。

10.5 示例五：反应扩散方程

```
1 import numpy as np
2 from MosicaFE import FESpace, Mesh, ReactionDiffusionProblem
3
4 c = 10.0
5 exact = lambda x: np.sin(np.pi*x[0]) * np.sin(np.pi*x[1])
6 source = lambda x: (2*np.pi**2 + c) * exact(x)
7
8 mesh = Mesh.rectangle(nx=32, ny=32)
9 V = FESpace.lagrange(mesh, degree=2)
10 problem = ReactionDiffusionProblem(V, f=source, g=lambda x: 0.0,
11                                   reaction_coeff=c)
12 u = problem.solve()
13 print(problem.compute_error(u, exact, "L2"))    # ~6.8e-05
```

Listing 10.5: 示例 10.5: 内置的反应扩散问题

第十一章 进阶话题

11.1 精度陷阱与经验教训

把三套程序整合成一个库的过程中，实测暴露了四类问题。它们都不是“数值精度的小差别”，而是会给出错误结果或让收敛阶崩掉的结构性的错误。记录在此，既是为了说明 MosicaFE 为什么这么写，也供读者在自己的代码里自查。

11.1.1 陷阱一：Dirichlet 消去漏掉右端项修正

症状：齐次边界一切正常，非齐次边界解错。

诊断：用线性 patch test (式 (6.2))。精确解属于离散空间时，数值解必须精确。

结论：见式 (6.1)，第 (1) 步不可省。MosicaFE 把这条测试写进了 `tests/test_core_spaces.py`。

11.1.2 陷阱二：张量积单元的求积阶不足

症状：三维六面体 Q1 的 L^2 收敛阶算出来是 -0.11 。

诊断：Q1 刚度矩阵的被积函数在每个方向上是 2 次多项式，而每个方向只用了 1 个 Gauss 点时只能精确到 1 次，欠积分导致单元矩阵不是正定的、能量被系统性低估。

结论：张量积单元的求积阶取 $\max(2, 2*p)$ (每方向 p 个点起步)，见表格 4.5 后的注意事项。

11.1.3 陷阱三：虚拟元投影的常数模式规范化

症状：三角形与正方形网格上 VEM 收敛阶正常，但换到 Voronoi 多边形网格， L^2 收敛阶从 2 掉到 0.5 以下，误差在 10^{-2} 处停滞。

诊断：把线性精确解代入，检查数值解是否精确 (patch test)，发现不规则多边形上失败。

结论：常数模式必须按式 (3.12) 规范化，使投影精确再生一次多项式；否则稳定化项会惩罚线性函数，破坏相容性。

11.1.4 陷阱四：载荷 / 质量矩阵的几何权重

症状：即使修好了投影，不规则多边形网格上的 L^2 误差仍然偏大。

诊断：把 $\int_K \varphi_i$ 一律取成 $|K|/n$ ，相当于假设每个顶点“平分”单元。对三角形、正四边形这是对的，但对边长差异很大的多边形，短边上的顶点会被严重高估。

结论：改用式 (3.15) 的扇形线性插值，几何权重由单元自身形状决定。修复后 Voronoi 网格的 L^2 收敛阶回到 1.97。

11.1.5 陷阱五：numpy 的重复下标赋值

症状：`b[dofs] += local` 与稀疏矩阵组装结果不一致，质量矩阵的行和与载荷向量对不上。

诊断：`dofs` 中出现重复下标时（例如网格里有重复顶点），`numpy` 的缓冲赋值只保留最后一次，而 COO 组装会求和。

结论：一律使用 `numpy.add.at`。

11.2 网格细化策略

- 均匀加密：`mesh.refine_uniform()`，三角形单元数 $\times 4$ 、六面体 $\times 8$ ；
- 自适应加密（规划中）：基于残差型误差指示子

$$\eta_K = h_K \|f + \Delta u_h\|_{L^2(K)} + h_K^{1/2} \|[\partial_n u_h]\|_{L^2(\partial K)} \quad (11.1)$$

标记 $\eta_K > \theta \max_K \eta_K$ 的单元细分。MosicaFE 的 `Mesh` 支持变长单元，因此局部加密在数据结构上是可行的，缺的是 `refinement` 的实现。

11.3 性能与向量化

- (1) 组装阶段累计 COO 三元组，最后一次转成 CSR；
- (2) 参考单元上的基函数求值按”求积点”缓存（`LagrangeSpace._quadrature_data`），避免逐单元重复计算；
- (3) 对同一网格上多种物理问题，`stiffness_matrix()` 的结果可以复用；
- (4) 大规模问题建议改用迭代法；
- (5) 若要进一步提速，可以把单元循环用 `numba` 或”按单元类型分块 + 批量张量运算”的方式向量化。

11.4 与其它库的关系

MosicaFE 刻意保持”小、可读、可改”：

- 与 FEniCS / Firedrake 相比，MosicaFE 不是通用有限元平台，而是教学与研究用的可实现，代价是只支持结构化网格生成与有限几类单元；
- 与 `fealpy` 相比，MosicaFE 更聚焦于”FEM / VEM / RT0 三种离散方式的横向对比”；

- 与原始三套脚本相比，MosicaFE 把它们变成同一套接口、统一了网格与求解器、并补齐了验证。

第十二章 扩展框架

12.1 新增一种单元

步骤:

- (1) 新建 MosicaFE/spaces/yourspace.py, 继承 BaseFESpace;
- (2) 实现 `_build_dof_map()`: 建立单元局部自由度到全局自由度的映射, 并填写 `n_dofs`、`dof_support`、`dof_is_interior`;
- (3) 实现 `stiffness_matrix()`、`mass_matrix()`、`load_vector(f)`;
- (4) 实现 `interpolate(f)` 与 `evaluate(cell, u, points)`;
- (5) 在 `spaces/factory.py` 的 `FESpace.FAMILIES` 中注册;
- (6) 在 `tests/` 中补上 `patch test` 与收敛阶测试。

```
1 from MosicaFE.spaces.base import BaseFESpace
2
3 class MySpace(BaseFESpace):
4     family = "myspace"
5     has_explicit_basis = True
6
7     def _build_dof_map(self):
8         # fill self.dof_map, self.n_dofs, self.dof_support, self.dof_is_interior
9         ...
10
11     def reference_nodes(self, cell_idx):
12         # reference coordinates of local DOFs
13         ...
14
15     def eval_basis(self, xi):
16         ...
17
18     def eval_basis_grad(self, xi):
19         ...
20
```

```

21     def stiffness_matrix(self):
22         return self._assemble(self._local_stiffness, symmetric=True)
23
24     def mass_matrix(self):
25         return self._assemble(self._local_mass, symmetric=True)
26
27     def load_vector(self, f):
28         return self._assemble_vector(lambda c: self._local_load(c, f))
29
30     def interpolate(self, f):
31         return self._evaluate_function(f, self.dof_coords)

```

Listing 12.1: 示例 12.1: 显式基函数空间的骨架

要点

如果新单元的基函数是显式的（例如更高次的 Lagrange、DG、或张量积的其它族），可以参考 `spaces/lagrange.py`：它已经实现了通用的“参考单元求值 + 逐单元变换”组装循环，通常只需要提供 `eval_basis` 与自由度布局。

12.2 新增一门方程

```

1  from MosicaFE.physics.base import BasePhysics
2  from MosicaFE.physics.registry import register_problem
3
4  @register_problem("my_pde")
5  class MyProblem(BasePhysics):
6      """Weak form:  $a(u, v) = l(v)$ ."""
7
8      def __init__(self, space, f=None, g=None, **kwargs):
9          super().__init__(space, f=f, g=g, **kwargs)
10         # store problem specific data here
11
12     def assemble(self):
13         K = self.space.stiffness_matrix()
14         M = self.space.mass_matrix()
15         A = K + 3.0 * M  # example combination
16         b = self.space.load_vector(self.f)
17         return self.apply_boundary_conditions(A, b)

```

Listing 12.2: 示例 12.2: 新 PDE 模板

注册之后，`available_problems()` 会包含 "my_pde"，并且 `create_problem("my_pde", V, ...)` 与 `run("my_pde", mesh, element="VEM", ...)` 立刻可用。

12.3 新增一种混合格式（以 $RT_0 \times P_0$ 为模板）

混合格式（鞍点问题）与上面的“椭圆算子”套路不同，但分工是一样的：空间算出单元矩阵块，物理层只做拼装。 $RT_0 \times P_0$ 在库内就是按这个模板实现的，可以照抄：

- (1) 如果需要新的面自由度布局,先在 `core/facets.py` 拿到面拓扑(全局面编号、`cell_facets`、`signs`);
- (2) 在空间里实现”块”接口:本例是 `flux_mass_matrix()` ((Φ, Φ)) 与 `divergence_matrix()` ((P, Φ)), 外加 `pressure_load_vector(f)`;
- (3) 在物理层用 `bmat` (来自 `MosicaFE._compat`) 把块拼成鞍点矩阵: `bmat([[M, -B.T], [B, None]])`;
- (4) 求解方式: "direct"、"schur" (静力凝聚) 或 `Solver.minres` (对称不定);
- (5) 误差分析放进 `postprocess/error.py`, 参照 `mixed_error_norms` (压力 + 通量各一套范数)。

要点

`physics` 层只有 6 行拼装代码 (见 `physics/mixed_poisson.py` 的 `assemble`), 而单元几何与数值积分全部留在 `spaces/rt0.py`。因此换一套混合元 (例如 BDM1、或 Stokes 的 $Q_2 \times Q_1$ 配对) 时, 只需要提供新的块, 物理层与后处理可以原样复用。

12.4 新增一种网格

- (1) 在 `core/mesh.py` 中新增一个类方法, 返回 `Mesh(nodes, elements, element_type, name=...)`;
- (2) 2D 单元会自动按带符号面积纠正为逆时针; 3D 四面体会按体积纠正取向;
- (3) 如果 3D 单元不是四面体 / 六面体, 请通过 `faces=...` 提供每个单元的面表 (局部顶点编号的列表)。

```

1 import numpy as np
2 from MosicaFE import Mesh
3
4 # a two-element hexagonal domain (arbitrary polygons are fine)
5 nodes = np.array([[0.0, 0.0], [1.0, 0.0], [1.5, 0.9],
6                   [0.5, 1.5], [-0.5, 0.9], [0.5, 0.6]])
7 elements = [[0, 1, 2, 5], [0, 5, 3, 4],
8             [1, 2, 3, 5], [0, 3, 4, 5]]
9
10 mesh = Mesh.from_arrays(nodes, elements, boundary_nodes=[0, 1, 2, 3, 4])
11 print(mesh.summary())

```

Listing 12.3: 示例 12.3: 自定义单元形状

12.5 开发流程与测试要求

建议的流程:

- (1) 先写失败的测试 (patch test、收敛阶、对称性等);
- (2) 再实现功能;
- (3) 跑 `tests/run_tests.py` 与 `tests/run_checks.py`;
- (4) 更新 `progress/` 下的进度与验证记录。

每个新特性至少要覆盖:

- 单元级: 基函数的 Kronecker 性质、单位分解、单元矩阵对称性/正定性;
- 空间级: 全局矩阵的对称性、常数核、 M 行和与 $b(f \equiv 1)$ 的一致性;
- 问题级: patch test (线性精确解必须精确)、收敛阶;
- 端到端: `solve_poisson / run` 一行接口可用。

第十三章 常见问题与排查

13.1 常见问题

13.1.1 导入错误

现象: `ModuleNotFoundError: No module named 'MosicaFE'`。

原因: MosicaFE 在 `D:\mosica\MosicaFE`, 需要把父目录 `D:\mosica` 加入模块路径。

```
# option 1: install in editable mode
cd D:\mosica\MosicaFE && python -m pip install -e .

# option 2: extend PYTHONPATH
$env:PYTHONPATH = "D:\mosica"
```

13.1.2 收敛阶不对

排查顺序:

- (1) 先跑 `patch test` (第六章的式 (6.2))。若不通过, 问题一定在组装或边界处理;
- (2) 检查求积阶是否足够 (张量积单元尤其容易欠积分);
- (3) 检查 h 的定义是否与网格加密一致;
- (4) 检查精确解是否足够光滑 (制造解最好用三角函数或多项式);
- (5) 对 VEM, 检查稳定化选项是否为 `"trace"`。

13.1.3 迭代法不收敛

- 先用直接法验证系统本身可解;
- CG 要求矩阵对称正定: 鞍点问题必须换 MINRES 或直接法;
- 增大 `maxiter`、放宽 `tol`;
- 条件数很大时考虑缩放或预处理。

13.1.4 内存不足

- 确认在使用稀疏矩阵（安装了 `scipy`）；
- 三维问题网格数增长很快： n^3 个节点对应约 $6n^3$ 个四面体；
- 改用迭代法可避免直接法的填充（fill-in）。

13.2 常见问题

问：怎样选择多项式次数？ 答：绝大多数情形 P2 / Q2 的精度/开销比最好；先用 P1 做快速验证，解很光滑时再上二次元。虚拟元目前只有 $k = 1$ 。

问：什么时候用迭代法？ 答：自由度少于约 2×10^4 时直接法最省心；更大规模改用 CG（对称正定）或 GMRES（一般）。

问：VEM 与 FEM 到底差在哪？ 答：在三角形网格上， $k = 1$ 虚拟元空间恰好等于 P1 有限元空间，两者数值结果完全相同（本库实测两者误差逐位一致）。VEM 的优势出现在一般多边形网格上：它允许每个单元的面数不同、允许非结构的多边形剖分，而经典 Lagrange 有限元需要单元形状统一。

问：为什么 RT0 的解返回两个数组？ 答：混合格式同时求解通量 q_h 与压力 p_h 。Q[c,i] 是单元 c 第 i 个面（按“不含顶点 i ”编号）上的外法向通量，P[c] 是该单元的常数压力。

问：RT0 需要安装或接入外部程序吗？ 答：不需要。RT0×P0 已经是 MosicaFE 自己的方法：显式基函数 $\varphi_i = (x - a_i)/(d|K|)$ 与精确质量矩阵在 `spaces/rt0.py`，面拓扑与定向在 `core/facets.py`，鞍点组装在 `physics/mixed_poisson.py`，误差范数在 `postprocess/error.py`。库的依赖仍然只有 `numpy` (`scipy` / `matplotlib` 可选)，`import MosicaFE` 之后 `FESpace.rt0(mesh)` 直接可用。

问：能求解时间相关问题吗？ 答：当前版本只处理稳态椭圆问题。时间相关问题建议用“线方法”：空间离散得到 $M\dot{u} + Au = b$ ，时间方向自行用显式/隐式格式推进。`mass_matrix()` 已经提供。

问：能求解非线性问题吗？ 答：框架面向线性问题。非线性问题建议在外层写 Newton 迭代，每一步用 MosicaFE 组装雅可比并求解。

问：怎样把结果导出到 ParaView？ 答：用 `space.nodal_values(u)` 取得节点值，配合 `mesh.nodes` 与 `mesh.elements` 自行写 `.vtu` 或 `.vtk`；也可以在 `postprocess` 中新增导出函数。

附录 A API 速查

A.1 顶层模块 MosicaFE.

名字	说明
Mesh	网格对象与工厂方法
FESpace	空间工厂: <code>lagrange</code> / <code>vem</code> / <code>rt0</code> / <code>create</code>
FacetTopology, build_facet_topology	面拓扑与定向 ($H(\text{div})$ 空间)
QuadratureRule	参考单元求积规则
cell_quadrature(...) / simplex_quadrature(...)	物理单元 / 任意单纯形上的求积点与权重
BaseFESpace	空间基类
LagrangeSpace / VEMSpace / RT0Space	三种空间
rt0_flux_basis, rt0_mass_matrices	RT0 显式基函数与精确质量矩阵
BasePhysics	物理问题基类
PoissonProblem	$-\nabla \cdot (\kappa \nabla u) = f$
MixedPoissonProblem	RT0×P0 混合形式
ReactionDiffusionProblem	$-\Delta u + cu = f$
register_problem, create_problem, available_problems	PDE 注册表
Solver	线性求解器 (<code>direct</code> / <code>cg</code> / <code>gmres</code> / <code>bicgstab</code> / <code>minres</code> / <code>auto</code>)
compute_error / compute_errors	误差范数
mixed_error_norms	RT0×P0 的压力 / 通量误差范数
convergence_study	收敛性研究
estimate_convergence_rate	收敛阶估计
plot_solution / plot_mesh / plot_error	可视化
plot_convergence / plot_pressure_2d / plot_flux_2d	可视化
solve_poisson	一行式 Poisson 求解

名字	说明
run	按名字求解任意已注册 PDE

A.2 Mesh

方法 / 属性	说明
<code>Mesh.interval(a,b,nx)</code>	一维区间网格
<code>Mesh.rectangle(...)</code>	二维矩形网格 (<code>triangle/quad</code>)
<code>Mesh.box(...)</code>	三维长方体网格 (<code>tet/hex</code>)
<code>Mesh.pentagon(nx,ny)</code>	五边形网格
<code>Mesh.voronoi_polygon(...)</code>	Voronoi 多边形网格
<code>Mesh.from_arrays(...)</code>	自定义网格
<code>mesh.refine_uniform()</code>	均匀加密
<code>mesh.get_mesh_size()</code>	最大单元直径 h
<code>mesh.get_cell_vertices(i)</code>	单元顶点坐标
<code>mesh.get_cell_measure(i)</code>	单元面积 / 体积
<code>mesh.get_cell_diameter(i)</code>	单元直径
<code>mesh.get_cell_centroid(i)</code>	单元形心
<code>mesh.boundary_nodes</code>	边界顶点编号
<code>mesh.boundary_facets /</code> <code>mesh.facets</code>	facet 拓扑
<code>mesh.facet_topology</code>	FacetTopology: 全局面编号、共享面、外法向符号
<code>mesh.summary()</code>	一行摘要

A.3 FESpace 与空间方法

方法	说明
<code>FESpace.lagrange(mesh, degree)</code>	连续 Lagrange ($P1/P2/Q1/Q2$)
<code>FESpace.vem(mesh, degree=1)</code>	虚拟元 ($k = 1$)
<code>FESpace.rt0(mesh)</code>	$RT0 \times P0$ 混合空间
<code>FESpace.create(mesh, family, degree)</code>	按名字创建
<code>FESpace.available()</code>	可用空间族
<code>V.n_dofs</code>	自由度个数
<code>V.get_cell_dofs(i)</code>	单元局部 $\rightarrow$ 全局自由度

方法	说明
<code>V.get_dof_coordinates(k)</code>	自由度坐标
<code>V.boundary_dofs()</code>	Dirichlet 自由度
<code>V.eval_basis(xi) /</code> <code>V.eval_basis_grad(xi)</code>	参考单元基函数
<code>V.stiffness_matrix()</code>	刚度矩阵
<code>V.mass_matrix()</code>	质量矩阵
<code>V.load_vector(f)</code>	载荷向量
<code>V.interpolate(f)</code>	插值
<code>V.evaluate(c, u, pts),</code> <code>V.evaluate_gradient(...)</code>	单元内重构求值
<code>V.nodal_values(u)</code>	节点值（可视化用）
<code>V.flux_mass_matrix()</code>	RT0: (Φ, Φ) 块（精确闭式质量矩阵）
<code>V.divergence_matrix()</code>	RT0: (P, Φ) 块（散度，恒为 ± 1 ）
<code>V.pressure_load_vector(f)</code>	RT0: 压力方程右端项 $\int_K f$
<code>V.interpolate_pressure(f) /</code> <code>V.interpolate_flux(q)</code>	RT0: 两个场各自的插值
<code>V.evaluate_flux(c, Q, pts)</code>	RT0: 单元内重构 q_h
<code>V.locate_cell(x)</code>	RT0: 定位点所在单元
<code>V.n_flux / V.n_pressure /</code> <code>V.topology</code>	RT0: 自由度规模与面拓扑

A.4 物理问题

方法	说明
<code>PoissonProblem(V, f, g, kappa)</code>	构造
<code>problem.assemble()</code>	返回 (A, b) （含边界条件）
<code>problem.solve(method)</code>	组装并求解
<code>problem.compute_error(u, exact, norm)</code>	单个误差范数
<code>problem.compute_errors(u, exact, grad)</code>	全部误差范数
<code>MixedPoissonProblem(...)</code>	返回 (Q, P)
<code>mixed.error_norms(P, Q, p_exact, q_exact)</code>	压力/通量误差
<code>mixed.check_conservation()</code>	单元质量守恒误差
<code>mixed.flux_at_points(points)</code>	给定点上的通量重构（自动定位单元）

方法	说明
<code>mixed.cell_flux(cell)</code>	单元形心处的通量向量（画箭头用）
<code>mixed.solve(method)</code>	鞍点系统的三种求解方式： <code>direct</code> / <code>schur</code> / <code>minres</code>

A.5 求解器与后处理

方法	说明
<code>Solver.direct(A,b)</code>	直接法
<code>Solver.conjugate_gradient(...)</code>	CG
<code>Solver.gmres(A,b,tol)</code>	GMRES
<code>Solver.bicgstab(A,b,tol)</code>	BiCGSTAB
<code>Solver.minres(A,b,tol)</code>	MINRES
<code>Solver.solve(A,b,method)</code>	统一入口
<code>Solver.apply_dirichlet(...)</code>	强加 Dirichlet
<code>Solver.recommend(n, symmetric)</code>	推荐求解器
<code>convergence_study(...)</code>	收敛性研究
<code>estimate_convergence_rate(h, e)</code>	收敛阶
<code>plot_solution(u, mesh, V, ...)</code>	画解
<code>plot_mesh(mesh, ...)</code>	画网格
<code>plot_error(u, mesh, space, exact)</code>	画误差
<code>plot_convergence(h, errors)</code>	画收敛曲线
<code>plot_pressure_2d(mesh, P)</code>	画 RT0 压力
<code>plot_flux_2d(space=V, Q=Q)</code>	画通量（用 q_h 在单元形心处重构）
<code>compare_solutions(entries)</code>	并排比较

附录 B 数学符号

B.1 一般记号

符号	含义
Ω	求解区域
$\partial\Omega$	区域边界
d	空间维数 (1、2、3)
h	网格尺寸 (最大单元直径)
K	网格单元
$\hat{K}$	参考单元
n	自由度个数
$\mathcal{D}$	Dirichlet 自由度集合

B.2 函数空间

符号	含义
V	无穷维试探/检验空间
$V_h \subset V$	有限维离散空间 (有限元或虚拟元)
$H^1(\Omega)$	一阶 Sobolev 空间
$H(\text{div}; \Omega)$	散度平方可积的向量场空间
$L^2(\Omega)$	平方可积函数空间
$\mathcal{P}_k$	次数不超过 k 的多项式空间
Q_k	每个方向次数不超过 k 的张量积多项式空间

B.3 算子与范数

符号	含义
∇u	梯度
$\nabla \cdot q$	散度
Δu	Laplace 算子
$\varphi_i / \hat{\varphi}_i$	物理 / 参考单元上的基函数
Π^∇	VEM 的梯度投影算子
Π_{dof}	投影算子在自由度上的矩阵表示
$A_{\text{proj}}, S_{\text{stab}}$	VEM 单元矩阵的一致性项与稳定化项
J_K	参考单元到物理单元的雅可比矩阵
$\ e\ _{L^2}, e _{H^1}, \ e\ _{L^\infty}$	误差范数
q, p	RT0 混合格式中的通量与压力
A, b	离散线性系统的矩阵与右端项

附录 C 参考文献

参考文献

- [1] S. C. Brenner and L. R. Scott, *The Mathematical Theory of Finite Element Methods*, 3rd ed., Springer, 2008.
- [2] A. Ern and J.-L. Guermond, *Theory and Practice of Finite Elements*, Springer, 2004.
- [3] A. Logg, K.-A. Mardal, and G. Wells (eds.), *Automated Solution of Differential Equations by the Finite Element Method*, Springer, 2012.
- [4] L. Beirão da Veiga, F. Brezzi, A. Cangiani, G. Manzini, L. D. Marini, and A. Russo, “Basic principles of virtual element methods”, *Mathematical Models and Methods in Applied Sciences*, 23(1):199–214, 2013.
- [5] L. Beirão da Veiga, F. Brezzi, L. D. Marini, and A. Russo, “The Hitchhiker’s Guide to the Virtual Element Method”, *Mathematical Models and Methods in Applied Sciences*, 24(8):1541–1573, 2014.
- [6] L. Beirão da Veiga, F. Brezzi, L. D. Marini, and A. Russo, “Virtual element methods for general second order elliptic problems on polygonal meshes”, *SIAM Journal on Numerical Analysis*, 2013.
- [7] F. Brezzi and M. Fortin, *Mixed and Hybrid Finite Element Methods*, Springer, 1991.
- [8] P.-A. Raviart and J.-M. Thomas, “A mixed finite element method for 2nd order elliptic problems”, in *Mathematical Aspects of Finite Element Methods*, Springer, 1977.
- [9] Y. Saad, *Iterative Methods for Sparse Linear Systems*, 2nd ed., SIAM, 2003.